PROJET ELK
Document de synthèse

Par

Esli Othniel KADJO

Theo SAMINATDIN

Dris MOUSSAOUI

Made with GAMMA

1. Contexte, objectifs et périmètre

Problème traité

Le secteur du cinéma génère une quantité massive de données : titres, genres, budgets, recettes, notes, langues, dates de sortie, etc. Ces données sont souvent issues de sources hétérogènes (fichiers plats, différentes API) et nécessitent une infrastructure capable de les ingérer, les nettoyer et les rendre interrogeables de manière rapide et visuelle. C'est précisément le problème que ce projet cherche à résoudre.

Objectifs techniques et métier

L'objectif technique principal est de construire une plateforme de données Free complète et reproductible, reposant sur la stack ELK (Elasticsearch, Logstash, Kibana). Concrètement, cela signifie ingérer un dataset de films au format CSV, le nettoyer et le structurer via Logstash, l'indexer dans Elasticsearch avec deux formes brutes et nettoyées puis produire des visualisations exploitables dans Kibana.

Sur le plan métier, la plateforme doit permettre de répondre à des questions business telles que : quels sont les films les mieux notés (et par combien) ? Quels sont les films sortis après 2010 ? Quel est le top 10 des films les plus présentés ? Un mini-moteur de recherche textuelle sur les titres et les synopsis est également envisagé comme extension. Ces questions constituent des embarquées possibles identifiées mais hors du cadre du projet initial.

Les trois ELK en quelques mots

L'architecture repose sur quatre services conteneurisés, orchestrés via Docker Compose. Chaque service a un rôle précis et communique avec les autres via le réseau interne créé par Compose. Le démarrage se fait via Elasticsearch, puis Kibana attend qu'il soit sain (healthcheck) et disponible avant de s'initialiser. Cette dépendance est encodée garantit que les services dérivant dans le bon ordre à chaque (environ) système au lancement.

```
movies.csv
|
Logstash → lit, nettoie, transforme
|
Elasticsearch → stocke dans movies_raw puis movies_clean
|
Kibana → visualise et explore
|
Jupyter → analyse et exploration Python
```

Description du docker compose

Le fichier docker compose.yml définit quatre services tous basés sur la version 8.11.0 de la stack Elastic. Elasticsearch est configuré en mode réseau unique (single-node) avec la sécurité désactivée et le security disabled pour un usage local simplifié. Il expose le port 9200 pour les requêtes REST. Kibana tout à Elasticsearch via la variable d'environnement ELASTICSEARCH_HOSTS et expose son interface web sur le port 5601. Logstash partage le même dossier local des données que le conteneur Elasticsearch. Le fichier de pipeline logstash.conf, le dossier de données contenant le CSV et le dossier de logs. Jupyter contient également les mêmes volumes en lecture seule : le fichier de pipeline logstash.conf, le dossier de données contenant le CSV et le dossier de logs. Jupyter contient également les mêmes volumes en lecture seule : le fichier de pipeline logstash.conf, le dossier de données contenant le CSV et le dossier de logs. Jupyter contient également les mêmes volumes en lecture seule : le fichier de pipeline logstash.conf, le dossier de données contenant le CSV et le dossier de logs. Jupyter contient également les mêmes bibliothèques pandas et matplotlib pré-installées. Il est accessible sur le port 8888.

Trois services dépassent d'un healthcheck. Chaque conteneur vérifie régulièrement chaque service pour s'assurer qu'il répond correctement avant d'autoriser les services dépendants à démarrer.

L'ensemble du projet est organisé dans un dossier principal Volumes. Le dossier contient trois éléments nécessaires au fonctionnement du pipeline : le fichier docker-compose.yml qui définit l'ensemble des services, le dossier logstash/ qui contient la configuration du pipeline, et le dossier data/ qui contient le fichier source movies.csv.

Les fichiers de configuration Docker sont placés dans un dossier dédié plutôt qu'à la racine de ce dossier. Le dossier composant doit fonctionnelle des services et leurs relations. Seuls le Dockerfile/jupyter contient les instructions pour construire l'image Jupyter personnalisée. Le fichier requirements.txt liste les dépendances Python à installer dans cette image. notamment elasticsearch, pandas et matplotlib.

Les données du projet sont stockées dans un sous-dossier data qui contient le fichier Movies.csv. Ce dossier est monté en lecture seule dans le conteneur Logstash afin que les données sources ne puissent jamais être modifiées accidentellement par un pipeline mal configuré. Le dossier contient aussi le CSV lui-même et quelques dérivés de parsing.

La configuration de Logstash est placée dans un sous-dossier logstash/ séparé qui contient le fichier logstash.conf. Créé dans un fichier unique et indépendant des autres règles de lecture, de nettoyage et d'envoi des données vers Elasticsearch. Il est lui aussi monté en lecture seule dans le conteneur pour que la configuration ne puisse pas être altérée pendant l'exécution.

Enfin, deux dossiers vides sont présents dans le dossier racine : logs/ et notebooks/. Logs/ regroupe les journaux produits par Logstash et d'éventuels scripts de monitoring, et notebooks/ est prêt à recevoir des fichiers .ipynb. Ces dossiers doivent exister sur la machine avant le lancement de la stack, mais leur contenu n'est pas versionné dans Git.

Plus de données de bout en bout

Le fichier Movies.csv est monté en lecture seule dans le conteneur Logstash via un volume Docker. Logstash lit ce fichier ligne par ligne, découpe chaque ligne selon les colonnes du CSV, applique des transformations (conversion de types, nettoyage des champs vides, normalisation des listes) puis envoie les documents vers Elasticsearch. Deux index sont alimentés : movies_raw reçoit les données brutes telles que lues, sans aucune modification, et movies_clean reçoit les données après application complète des règles de nettoyage. Kibana se connecte ensuite à Elasticsearch pour créer des visualisations et explorer les données de movies_clean. Jupyter permet en parallèle d'interroger Elasticsearch en Python pour des analyses complémentaires.

2. Données et nettoyage

Description des colonnes

Le fichier de données "Movies.csv" contient un total de 26 colonnes. Chaque colonne jouer un rôle différent dans l'analyse des données et leur transformation. Voici une présentation détaillée des colonnes principales :

- title : Colle colonne contient le titre du film. Elle est essentielle pour identifier chaque entrée de manière unique. Les doublons dans cette colonne peuvent indiquer des erreurs ou des versions multiples d'un même film.
- genre : Cette colonne indique la ou les genres du film (par exemple, "Action", "Comédie"). Elle est utilisée pour catégoriser les films et analyser les tendances par genre.
- release_date : La date de sortie du film. Cte chemp est crucial pour analyse les tendances temporelles et identifier les films sortis dans une période donnée. Les données peuvent être parfois incomplètes, dans le futur ont cela compte.
- budget : Le budget de production du film, exprimé en dollars. Les valeurs manquantes ou aberrantes (par exemple, un budget de 1 pour un blockbuster) peuvent fausser les analyses économiques. Ce champ est critique.
- revenue : Les revenus générés par le film, également en dollars. Ce champ est utilisé pour calculer le retour sur investissement (ROI) et analyser la rentabilité des films. Comme le budget, il peut être incomplet.
- runtime : La durée du film en minutes. Les valeurs extrêmes (par exemple, des durées inférieures à 10 minutes ou supérieures à 300 minutes) peuvent indiquer des erreurs ou des cas spéciaux.
- vote_average et vote_count : Ces colonnes contiennent respectivement la note moyenne et le nombre de votes reçus par le film. Elles sont utilisées pour évaluer la popularité et la qualité perçue des films.
- production_companies : Cette colonne liste les sociétés de production impliquées dans le film. Elle est utile pour analyser les tendances par studio ou groupe de production.

Anomalies détectées

Lors de l'analyse initiale des données, plusieurs anomalies ont été identifiées :

- Valeurs manquantes : Plusieurs colonnes, en particulier "revenue" et "budget".
- Données vides : Des "title" ou "release_date".
- Doublons dans les titres de films.

Ces anomalies ont été corrigées en appliquant des règles spécifiques de nettoyage.

Règles Logstash appliquées

Pour intégrer les données dans Elasticsearch, des règles Logstash ont été définies :

- Ligne drop-table : Exclure toute ligne où title = "id" (drop { }).
- Données incomplètes : Les lignes sans titre, date de sortie ou budget seront supprimées.
- Champs parasites : Les champs techniques ajoutés par Logstash sont supprimés pour allager les index.

Comparaison avant/après

Avant le nettoyage, les données contiennent de nombreuses incohérences et doient indifférentes pour une analyse appropriée. Après application des règles de nettoyage et de transformation, les données sont deuxnues cohérentes, complètes et prêtes à être indexées dans Elasticsearch.

3. Modélisation Elasticsearch

Mapping de movies_clean

Le mapping de l'index movies_clean a été conçu pour optimiser les performances de recherche et d'analyse dans Elasticsearch. Voici les principaux choix effectués :

- Champs text/keyword :
  - Les champs comme title et overview sont définis comme "text" pour permettre des recherches en texte intégral.
  - Les champs comme original_language, status et genres sont définis comme "keyword" pour des recherches exactes et des agrégations.
- Champs numériques et dates :
  - Les champs budget, revenue, runtime, vote_average, et vote_count sont définis comme des types numériques ("float" ou "integer") pour permettre des calculs et des agrégations.
  - Le champ release_date est défini comme un type "date" avec le format yyyy-MM-dd .

Analyseur personnalisé

Un analyseur personnalisé a été créé pour le champ title afin d'optimiser les recherches textuelles. Cet analyseur inclut :

- Un tokenizer standard pour découper les mots.
- Un filtre de type "lowercase" pour ignorer les différences entre majuscules et minuscules.
- Un filtre de suppression des caractères spéciaux pour uniformiser les données.

Justification

Ces choix permettent d'équilibrer les besoins en recherche textuelle et en agrégation :

- Les champs "text" permettent des recherches avancées, comme la recherche par mots-clés dans les titres ou les descriptions.
- Les champs "keyword" facilitent les regroupements et les filtres dans Kibana, par exemple pour analyser les films par langue ou par genre.
- Les analyseurs personnalisés garantissent des résultats de recherche pertinents en normalisant les données textuelles.

En résumé, le mapping et les analyseurs mêlent rigueur et souplesse, ainsi nous pouvons pour répondre aux besoins spécifiques du projet, en maximisant la pertinence des recherches et l'efficacité des analyses.

4. Requêtes et analyses

Dans le cadre de la section 4, nous avons produit 5 requêtes Elasticsearch documentées sur l'index movies_clean, dont 3 requêtes bool. Ces requêtes couvrent différents cas d'usage métier: recherche, filtrage, tri et agrégation.

Requêtes de recherche et exploration

Les premières requêtes permettent de rechercher des films et d'explorer. Un simple GET /movies_clean/_count retourne 351 352 documents indexés, ce qui confirme la bonne ingestion des données. Une requête match_all avec size: 5 permet de visualiser la structure d'un document et de vérifier que les champs sont bien typés (date, float, keyword, text).

La recherche full-text fonctionne correctement sur les champs title et overview. Par exemple, une recherche sur "Batman" dans le titre retourne les films correspondants triés par pertinence.

Requêtes bool

Les requêtes bool permettent de combiner plusieurs conditions. Voici les 3 requêtes bool réalisées :

- Q5 : Films en anglais avec note ≥ 7 combinant un must sur la langue et un filter sur la note.
- Q7 : Films du genre Action sortis après 2010, avec un must sur le genre et un filter sur la date.
- Q8 : Films populaires (popularity > 50) mais des mauvaises critiques (vote_average < 5), avec un must sur la langue anglaise.
- Q9 : Films avec un budget supérieur à 100M€ et des revenus supérieurs à 200M€, en combinant deux filter de type range.
- Q10 : Films Drama ou Comedy avec vote_average >= 7.5, utilisant should avec minimum_should_match: 1.

Requêtes d'agrégation

Deux requêtes d'agrégation ont été produites pour analyser le dataset à fachter :

- Q6 : Agrégation terms sur original_language.keyword — résultat : langue dominante (environ 94% des films en anglais, suivi du japonais et du français).
- Q11 : Agrégation histogram sur vote_average.keyword et vote_average — résultat : les documentaires et les films d'animation obtiennent les meilleures notes moyennes.

Dashboard Kibana et lecture métier

Nous avons conçu un dashboard nommé Dashboard_Projet_ELK sur Kibana avec 6 visualisations construites sur l'index movies_clean.

Visualisations réalisées

Metric : Nombre total de films indexés dans movies_clean, confirmant les 351 354 documents disponibles.

Pie chart : Répartition des films par langue (original_language). L'anglais représente la majorité écrasante du dataset, ce qui reflète la dominance du cinéma anglophone dans les données.

Bar chart vertical : Top des genres les plus fréquents. Drama, Comedy et Action ressortent comme les genres les plus représentés.

Line chart : Évolution du nombre de films par année. On observe une forte croissance à partir des années 2000, avec un pic autour de 2018-2020.

Bar chart horizontal : Distribution des notes (vote_average) = 10 le plus fréquent, suivi de notes entre 5 et 7 avec pic de films aux extrémes.

Tableau : Top 5 films les plus populaires avec leur score de popularité. The Northman arrive en tête avec 3 855 points.

Limites d'interprétation

La popularité dans un dataset est une métrique calculée par TMDB et ne reflète pas nécessairement le box-office réel. De plus, les films avec une note de 0 ou 10 ne sont pas forcément à dénigrer les films sans votes, ce qui peut biaiser la distribution. Enfin, la surreprésentation des films anglophones limite l'analyse comparative entre les cinémas nationaux.

5. Gestion de projet et collaboration

Workflow Git structuré

Dès le début du projet, j'ai opté pour un workflow Git structuré afin de garantir un travail collaboratif propre et traçable. Le dépôt est organisé autour de trois niveaux de branches : main pour la version stable finale, dev pour l'intégration continue des features terminées, et des branches feature/* nommées selon le développement suivant de chacun fonctionnalité. Aucun push direct sur main ou dev n'est autorisé - chaque contribution passe obligatoirement par une Pull Request revue et approuvée au moins une fois avant d'être intégrée.

Dans les faits, ce workflow a bien été respecté tout au long du projet. Par exemple, lors de la PR de F1, la revoyer Thin Controllers, j'ai demandé des corrections précises : renommer le fichier controllers/movies.js logiquement cet qui les qui supprimant il n'y avait de inclus par erreur et ajouter des commentaires sur les routes. Ces retours ont été pris en compte et des commits correctifs ont été poussés avant le merge. De même, lors de la PR de F3, la revoyer a validé les trois critères attendus lecture du CSV, indexation dans movies_raw, et présence d'erreurs en console. Aucun et échantillon avant de merger. La PR de F3 a ainsi le mérite d'avoir mis en place une review d'ES MOUVO qui a permis de vérifier le parsing et le normalisation implémentés dans le logstash.conf.

Planning poker et estimation

Pour la répartition des tâches, j'ai adopté le planning poker en utilisant des cartes physiques collaborativement lorsque de chaque thème. Chaque membre a pris simultanément avec une valeur numérique, et les écarts de votes ont donné lieu à des échanges pour arriver au consensus. Cette méthode a bien fonctionné : l'effort total du projet a été estimé à 13 points, ce qui a permis de répartir les tâches de manière équilibrée, F1 et F2 ont pris chacune une fixtures du plus complexe, F3 et F4 ont pris en charge la qualité et la documentation (F3 à partir de la revoyer de recherche, F4 à partir de la revoyer des fixtures plus complexes. Il tient compte du fait que le dernier (F5 et D) regroupe toutes ES et des estimées à 2 points chacune, l'environnement étant bien balisé par le cours.

Répartition des fixtures

La répartition des fixtures a suivi les dépendances techniques naturelles du projet. Membres 1 et 2 ont chargé les fixtures F1, F2, Member 3 a implémenté les données et le moteur de recherche (F3), F4, et Member 4 a couvrir les tests et la documentation (F5). Cette répartition a permis à chacun de travailler sur des fonctionnalités indépendantes tout en restant proche du projet principal. Chacun a travaillé sur plusieurs fixtures en parallèle une fois le dev opérationnel : le nettoyage et le mapping d'un côté, le moteur de recherche de l'autre, ce qui a permis de maximiser les revues de la chaîne.

6. Bilan, limites et améliorations

Retour d'expérience

Le projet a permis de construire une chaîne de traitement de données complète, de la lecture d'un fichier CSV brut jusqu'à la visualisation sur deux index Elasticsearch. Le point le plus intéressant a été l'écart entre les attentes à la rédaction du cahier des charges (« il faut de l'analyse avec un vrai dataset ») et la réalité du terrain : le dataset initial a dû être remplacé par un autre, plus volumineux et plus hétérogène, ce qui a imposé de revoir le mapping et le pipeline Logstash en cours de route. Cette contrainte a finalement été bénéfique : elle a obligé à documenter le choix du mapping, vérifier la cohérence d'ingestion et rétrospectivement des notes, ce qui facilite la reproductibilité du projet.

Points bloquants rencontrés

Le principal point de friction a été le mapping du champ de l'volumineux Docker sur Windows. La gestion des chemins, notamment des espaces, a nécessité plusieurs tentatives avant de trouver une configuration stable. Le second point bloquant a été la gestion des complexes d'index. Ce problème a été corrigé en supprimant le CSV du dépôt et en ajoutant dans le README un bien sens le dataset original à téléchargeur manuellement. Cette contrainte impose tout de même un léger surcoût d'installation à quiconque veut le téléchargeur le fichier supérieur avant de lancer la stack.

Enfin, le workflow Git a nécessité une montée en compétence en début de projet. Des erreurs classiques ont été commises, comme commiter des fichiers appartenant à une autre feature ou inclure des modifications du logstash.conf de F2 dans la PR de F1. Des erreurs ont été détectées et corrigées grâce aux revues, ce qui confirme l'utilité du processus de review.

Limites identifiées

Le nettoyage des données reste perfectible sur certains champs. Les colonnes credits, keywords et recommendations contiennent des données complexes qu'il n'est pas actuellement normalisées ou simple split sur le séparateur. Une extraction structurée de ces champs (par exemple, le nombre de crédits ou le genre principal) ouvrirait des usages plus riches. Le moteur de recherche développé en F3 ignore les fonctionnalités fonctionnalités dernières mots ne propose pas encore de langue de scoring avancée ni de pagination.

Axes d'amélioration

Plusieurs pistes d'amélioration ont été identifiées. Sur le plan des données, un pré-traitement Python ou Jupyter permettrait de nettoyer plus finement les champs complexes avant l'ingestion, tout en conservant la simplicité du pipeline de base. Côté Kibana, le dashboard est fait, le plan de la visualisation Lens est prêt à être construit